

PONTIFÍCIA UNIVERSIDADE CATÓLICA DE GOIÁS
ESCOLA POLITÉCNICA E DE ARTES
CIÊNCIA DA COMPUTAÇÃO



**DOCUMENTAÇÃO TÉCNICA TRABALHO DE SISTEMAS DISTRIBUÍDOS – EQUIPE
PRODUTOS**

PEDRO MACEDO PARONETTO FARIA VIANA
RAIKY PREZOTTO PEREIRA SAHB NOVAES

GOIÂNIA
2025

1. MICROSERVIÇOS (INCLUINDO AUTENTICAÇÃO)

1.1 Requisitos de Negócio

Contexto do Domínio

O Portal B2B é uma plataforma de comércio eletrônico voltada à transação entre empresas (Business-to-Business). Fornecedores disponibilizam catálogos de produtos para compradores empresariais, que realizam pedidos com base nesses catálogos.

O microserviço de Produtos (SDI.Micro.Produto) é a fonte de verdade centralizada para informações descritivas de produtos. Ele não armazena preços nem estoques — essas responsabilidades pertencem ao domínio de Fornecedores e Inventário, seguindo o Bounded Context do DDD.

Problemas de Negócio Resolvidos

Problema	Solução
Ausência de catálogo centralizado e padronizado	Cadastro único com código e nome únicos em todo o sistema
Necessidade de classificar produtos por finalidade	Gerenciamento hierárquico de categorias com N níveis
Diferentes métodos de transporte por produto	Cadastro de modalidades de transporte associadas ao produto
Inconsistência de unidades de medida entre fornecedores	Unidades padronizadas com sigla normalizada (UPPERCASE)
Propagação de mudanças para outros serviços	Publicação de eventos via Kafka para sincronização assíncrona

Regras de Negócio Centrais

- Unicidade: nome, código, sigla são únicos no sistema (case-insensitive).
- Catálogo puro: sem preço nem quantidade — pertencem a outros domínios.
- Soft Delete: o campo ativo=FALSE substitui a exclusão física.
- Hierarquia de Categorias: auto-referência sem limite de profundidade.
- Auditoria: data_cadastro, usuario_cadastro, ultima_alteracao, usuario_alteracao em todo registro.
- Propagação via Eventos: toda alteração gera evento Kafka padronizado.

1.2 Jornada do Usuário

As etapas da jornada são: (1) Autenticação via JWT, (2) Configuração de pré-requisitos (transportes, categorias, unidades), (3) Cadastro do produto, (4) Consulta com filtros paginados, (5) Edição, (6) Inativação com publicação de evento.

Portal B2B - Diagrama de Arquitetura Geral

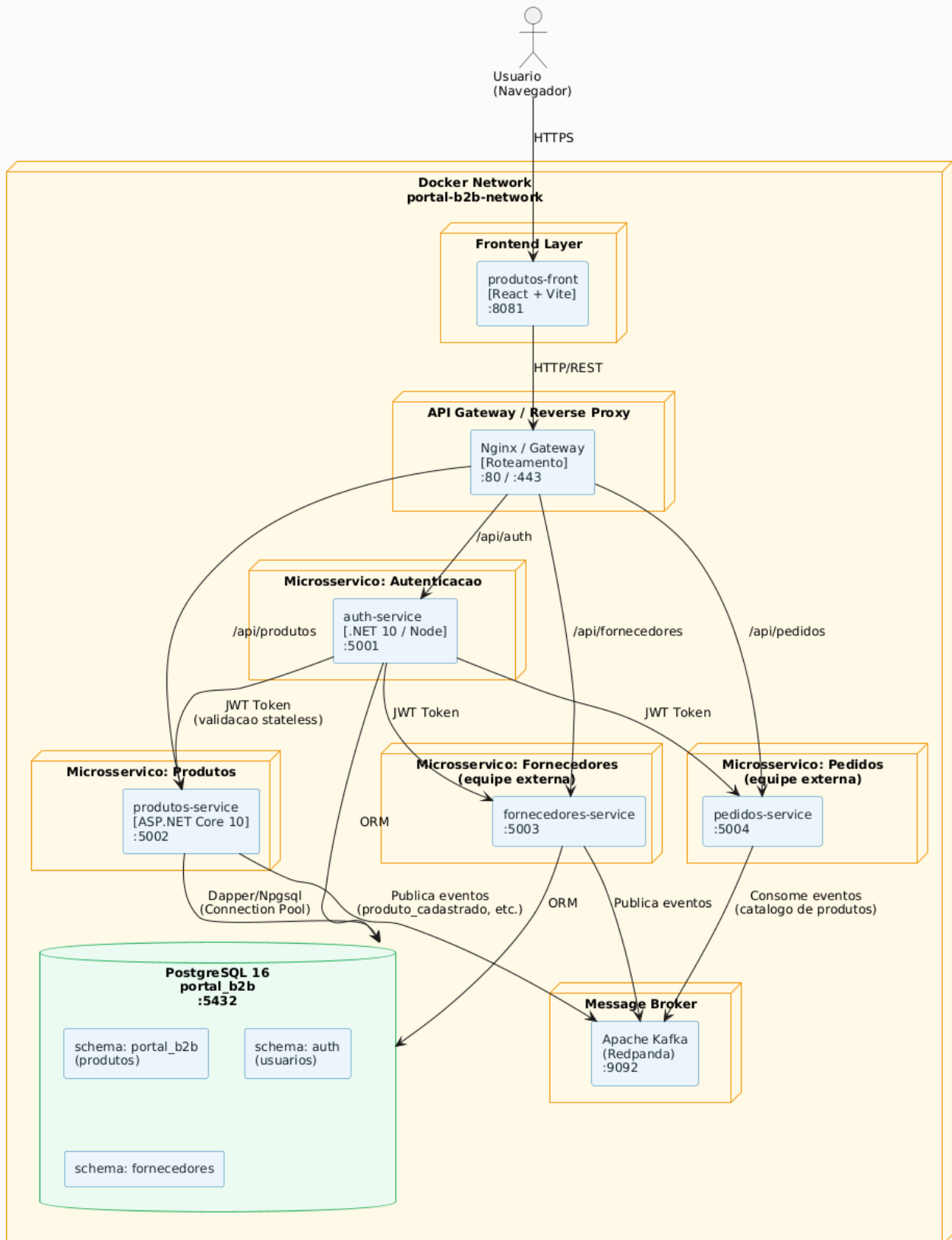


Figura 1 — Diagrama de Sequência: Jornada do Usuário

1.3 Protótipo (UX)

Tela	Título	Descrição
1	Login	Formulário e-mail/senha. Autentica via auth-service e redireciona ao dashboard.
2	Catálogo de Produtos	Tabela paginada com código, nome, categoria, transporte, unidade e status. Busca e filtros.
3	Novo/Editar Produto	Formulário com validação Zod em tempo real. Dropdowns de categoria, transporte, unidade.
4	Categorias	Lista hierárquica com subcategorias. CRUD completo com ativação/inativação.
5	Transportes	Listagem de modalidades de transporte. CRUD com ativação/inativação.
6	Unidades de Medida	Unidades com nome e sigla (UPPERCASE automático). CRUD completo.

1.4 Requisitos Funcionais

Código	Requisito
RF-01.1	Cadastrar produtos com código único (UPPERCASE), nome, descrição, categoria, transporte e unidade de medida
RF-01.2	Listagem paginada com filtros por busca textual, categoria, transporte, unidade e status
RF-01.3	Consulta de produto por ID
RF-01.4	Atualização completa dos dados
RF-01.5	Ativação e inativação (soft delete via PATCH)
RF-01.6	Publicação de evento Kafka em toda operação de escrita
RF-02.1	Categorias com hierarquia de N níveis (auto-referência)
RF-05.1	Todas as rotas (exceto /saude) exigem JWT Bearer
RF-05.3	O userId do JWT é registrado nos campos de auditoria
RF-06.1	GET /saude valida API e conectividade com PostgreSQL

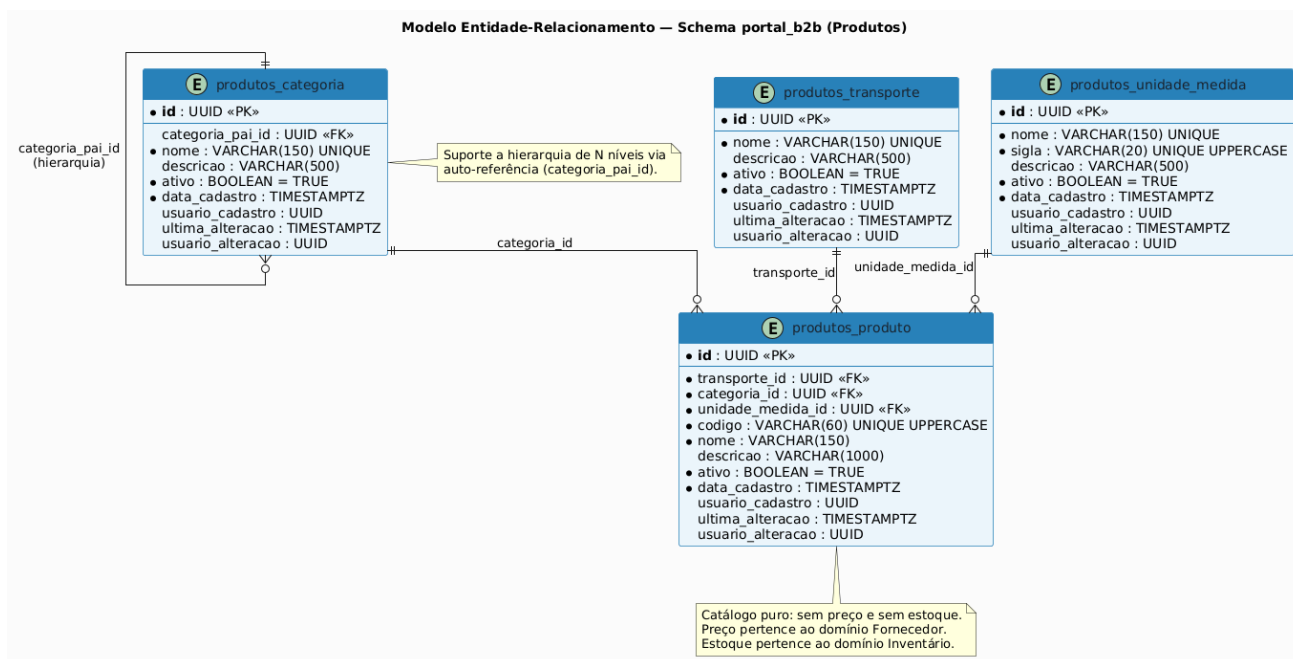


Figura 2 — Diagrama de Componentes: produtos-service (ASP.NET Core 10)

1.5 Requisitos Não Funcionais

Código	Categoria	Requisito
RNF-01.1	Desempenho	Latência < 100ms em leituras com índices otimizados
RNF-01.2	Desempenho	Pool de conexões PostgreSQL: até 100 conexões por instância
RNF-01.3	Desempenho	Paginação obrigatória em todas as listagens (máx. 100 itens/página)
RNF-02.1	Segurança	JWT com validação de issuer, audience, expiração e clock skew
RNF-02.3	Segurança	Queries parametrizadas (Dapper) — prevenção de SQL Injection
RNF-03.1	Disponibilidade	Restart policy unless-stopped no Docker Compose
RNF-03.3	Disponibilidade	Falha no Kafka não derruba a operação HTTP (FAIL_ON_PUBLISH_ERROR=false)
RNF-04.1	Observabilidade	Logging estruturado via Serilog (JSON) em todos os níveis
RNF-04.3	Observabilidade	Auditoria completa em todos os registros (4 campos de auditoria)

Tecnologias NÃO Utilizadas e Justificativas

Tecnologia	Motivo da Não Utilização
Entity Framework Core	Substituído por Dapper — maior controle sobre SQL e performance em queries complexas
Redis (Cache)	Não justificado: volume de dados moderado e necessidade de dados sempre atualizados
gRPC	Comunicação assíncrona via Kafka; REST adequado para frontend ↔ backend
RabbitMQ	Substituído por Apache Kafka (Redpanda) — decisão da equipe de infraestrutura
Oracle / SQL Server	Substituído por PostgreSQL (open-source, UUID nativo, decisão da equipe de infra)
Autenticação própria	Delegada ao microsserviço de autenticação — JWT validado de forma stateless

1.6 Tecnologias Utilizadas

Backend — ASP.NET Core 10

Tecnologia	Versão	Função
ASP.NET Core	10.0	Framework principal. DI nativo, Middleware Pipeline, JWT Bearer.
Dapper	2.1.66	Micro-ORM. SQL puro parametrizado com mapeamento snake_case → PascalCase.
Npgsql	9.0.3	Driver PostgreSQL. Connection pooling, UUID e TIMESTAMPTZ nativos.
Confluent.Kafka	2.8.0	Cliente Kafka oficial. Publica eventos de integração assíncronos.
Serilog	8.0.1	Logging estruturado em JSON. Integra com ELK Stack / Grafana Loki.
Swashbuckle	6.6.2	Documentação OpenAPI 3.0 gerada automaticamente.

Frontend — React + Vite

Tecnologia	Versão	Função
React	18.3.1	Biblioteca de UI declarativa com hooks modernos.
Vite	5.4.19	Build tool com HMR. Proxy reverso /api para o backend em dev.
TypeScript	5.8	Tipagem estática. Detecção de erros em compile-time.
Tailwind CSS	3.4	Framework CSS utilitário com purge automático no build.
Radix UI + shadcn/ui	latest	Componentes acessíveis (WAI-ARIA) customizáveis com Tailwind.

Tecnologia	Versão	Função
TanStack React Query	5.83	Cache de dados remotos com revalidação e retry automáticos.
Zustand	5.0.8	Estado global (JWT, usuário). Sem boilerplate excessivo.
React Hook Form + Zod	7.61.1	Formulários com validação de schema declarativa.
Axios	1.12.2	HTTP client com interceptors para Bearer Token automático.

1.7 Considerações Gerais

O produtos-service segue arquitetura em camadas: Controller → Service → Repository → PostgreSQL + Kafka. O Bounded Context de Catálogo exclui preços e estoques. O GlobalExceptionHandler converte exceções em respostas HTTP padronizadas (DomainException→400, UniqueViolation→409, FKViolation→400).

1.8 Instruções de Implantação

Componente	Versão Mínima
Docker Engine	24.0+
Docker Compose	2.20+
PostgreSQL	16 (na rede Docker)
Redpanda/Kafka	compat. Kafka 3.x

Passo 1 — Criar a rede Docker:

```
| docker network create portal-b2b-network
```

Passo 2 — Configurar .env:

```
| POSTGRES_HOST=postgres | Jwt__SecretKey=CHAVE |
| KAFKA_BOOTSTRAP_SERVERS=redpanda:9092
```

Passo 3 — Executar script de BD:

```
| psql -h localhost -U usuario -d portal_b2b -f Scripts/ScriptDeCriacao.sql
```

Passo 4 — Subir containers:

```
| docker compose up -d --build
```

Passo 5 — Verificar saúde:

```
| curl http://localhost:5002/saude
```

2. BANCO DE DADOS

2.1 Modelo de Dados Geral

O sistema utiliza PostgreSQL compartilhado (portal_b2b) com isolamento por schema. O microserviço de Produtos utiliza o schema portal_b2b com 4 tabelas.

Jornada do Usuário — Cadastro e Gestão de Produtos

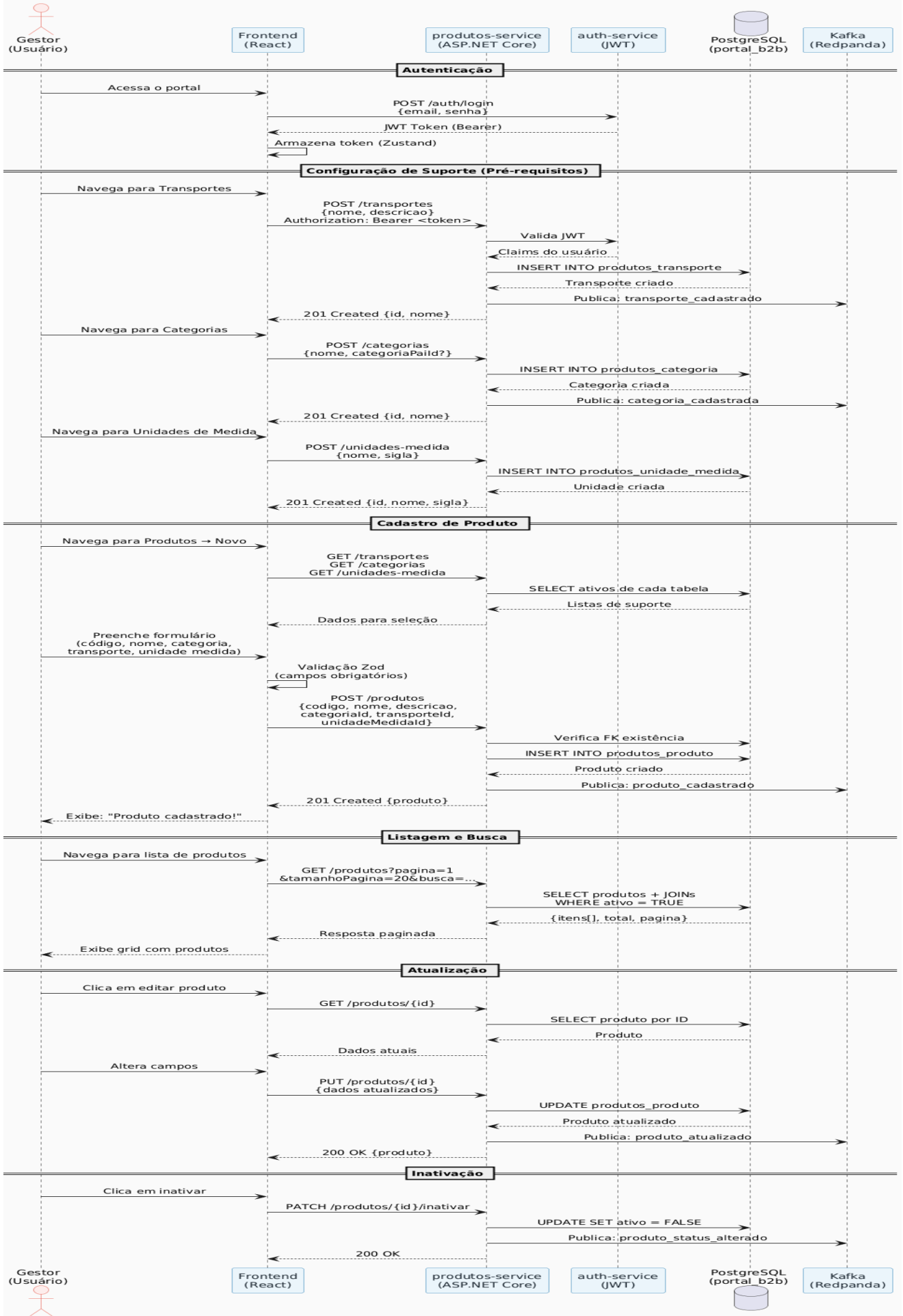


Figura 3 — Diagrama Entidade-Relacionamento: Schema portal_b2b (Produtos)

Tabela Origem	Coluna FK	Tabela Destino	Cardinalidade
produtos_produto	transporte_id	produtos_transporte.id	N:1
produtos_produto	categoria_id	produtos_categoria.id	N:1
produtos_produto	unidade_medida_id	produtos_unidade_medida.id	N:1
produtos_categoria	categoria_pai_id	produtos_categoria.id	N:1 (self)

2.2 Padrões de Nomenclatura

Todos os campos, tabelas e índices seguem snake_case. PostgreSQL converte identificadores para minúsculas por padrão; o snake_case evita necessidade de aspas duplas no SQL.

Mapeamento no .NET: DefaultTypeMap.MatchNamesWithUnderscores = true faz o Dapper mapear colunas snake_case para propriedades PascalCase automaticamente.

Padrão	Exemplo	Descrição
Chave primária	id	UUID gerado por gen_random_uuid()
Chave estrangeira	categoria_pai_id	{tabela_referenciada}_id
Status	ativo	Booleano para soft delete (padrão TRUE)
Data de criação	data_cadastro	TIMESTAMPTZ DEFAULT NOW()
Data de atualização	ultima_alteracao	TIMESTAMPTZ atualizado por trigger BEFORE UPDATE
Prefixo de tabela	produtos_*	Identifica o domínio/microserviço

2.3 Dicionário de Dados

Tabela: produtos_transporte

Campo	Tipo	Null	Descrição
id	UUID	NOT NULL	Identificador único — gen_random_uuid()
nome	VARCHAR(150)	NOT NULL	Nome único da modalidade (LOWER index)
descricao	VARCHAR(500)	NULL	Descrição opcional
ativo	BOOLEAN	NOT NULL	TRUE=ativo, FALSE=inativado. Padrão TRUE
data_cadastro	TIMESTAMPTZ	NOT NULL	Criação — DEFAULT NOW()
usuario_cadastro	UUID	NULL	ID do criador (extraído do JWT)
ultima_alteracao	TIMESTAMPTZ	NULL	Atualizado por trigger BEFORE UPDATE

Campo	Tipo	Null	Descrição
usuario_alteracao	UUID	NULL	ID do último alterador

Tabela: produtos_categoria

Campo	Tipo	Null	Descrição
id	UUID	NOT NULL	Identificador único
categoria_pai_id	UUID	NULL	FK auto-referencial — NULL = categoria raiz
nome	VARCHAR(150)	NOT NULL	Nome único (case-insensitive)
descricao	VARCHAR(500)	NULL	Descrição opcional
ativo	BOOLEAN	NOT NULL	Status. Padrão TRUE
data_cadastro	TIMESTAMPTZ	NOT NULL	Timestamp de criação
usuario_cadastro	UUID	NULL	ID do criador
ultima_alteracao	TIMESTAMPTZ	NULL	Trigger BEFORE UPDATE
usuario_alteracao	UUID	NULL	ID do último alterador

Tabela: produtos_unidade_medida

Campo	Tipo	Null	Descrição
id	UUID	NOT NULL	Identificador único
nome	VARCHAR(150)	NOT NULL	Ex.: "Quilograma". Único (case-insensitive)
sigla	VARCHAR(20)	NOT NULL	Ex.: "KG". Armazenada em UPPERCASE. Única
descricao	VARCHAR(500)	NULL	Descrição opcional
ativo	BOOLEAN	NOT NULL	Status. Padrão TRUE
data_cadastro	TIMESTAMPTZ	NOT NULL	Timestamp de criação
usuario_cadastro	UUID	NULL	ID do criador
ultima_alteracao	TIMESTAMPTZ	NULL	Trigger BEFORE UPDATE
usuario_alteracao	UUID	NULL	ID do último alterador

Tabela: produtos_produto

Entidade central. Catálogo puro — sem preço ou quantidade.

Campo	Tipo	Null	Descrição
id	UUID	NOT NULL	Identificador único do produto
transporte_id	UUID	NOT NULL	FK para produtos_transporte.id
categoria_id	UUID	NOT NULL	FK para produtos_categoria.id
unidade_medida_id	UUID	NOT NULL	FK para produtos_unidade_medida.id
codigo	VARCHAR(60)	NOT NULL	Código UPPERCASE único (case-insensitive)
nome	VARCHAR(150)	NOT NULL	Nome descritivo do produto
descricao	VARCHAR(1000)	NULL	Descrição detalhada opcional
ativo	BOOLEAN	NOT NULL	Status. Padrão TRUE
data_cadastro	TIMESTAMPTZ	NOT NULL	Timestamp de criação
usuario_cadastro	UUID	NULL	ID do criador
ultima_alteracao	TIMESTAMPTZ	NULL	Trigger BEFORE UPDATE
usuario_alteracao	UUID	NULL	ID do último alterador

2.4 Requisitos Não Funcionais — Banco Distribuído

Estratégia adotada: Shared Database, Separate Schema — banco PostgreSQL compartilhado com isolamento por schema. Cada microserviço conecta com usuário PostgreSQL com permissões restritas ao seu schema.

Aspecto	Estratégia
Isolamento	Schema separado por contexto (GRANT USAGE ON SCHEMA porta_b2b TO usuario_produtos)
Connection Pooling	Npgsql: até 100 conexões por instância (Maximum Pool Size=100)
Resiliência	Restart policy unless-stopped. Triggers de auditoria no banco (independem da app)
Sincronização	Via Kafka: cada serviço mantém cópia dos dados que precisa

2.5 Modelo por Microserviço

Todas as 4 tabelas pertencem ao schema portal_b2b do microserviço de Produtos:

- produtos_transporte (1) — (N) produtos_produto
- produtos_categoria (1) — (N) produtos_produto

- produtos_unidade_medida (1) — (N) produtos_produto
- produtos_categoria (1) — (N) produtos_categoria [hierarquia recursiva]

2.6 Tecnologias e Decisões Técnicas

PostgreSQL 16 escolhido por: UUID nativo (gen_random_uuid()), TIMESTAMPTZ com timezone, índices funcionais LOWER() para unicidade case-insensitive, ACID compliance e licença open-source.

Dapper escolhido em detrimento do EF Core: maior controle sobre SQL, performance superior em queries com JOINS complexos, debugging facilitado (SQL visível no código).

2.7 Conexão dos Microsserviços com o Banco de Dados

Hierarquia de conexão: NpgsqlDataSource (Singleton) → IDbConnectionFactory (Scoped) → NpgsqlConnection → Dapper → PostgreSQL :5432.

Resolução da connection string (prioridade): (1) variáveis de ambiente POSTGRESQL_* ; (2) ConnectionStrings:DefaultConnection no appsettings. O parâmetro Search Path=portal_b2b define o schema padrão, eliminando prefixo schema.tabela no SQL.

3. ARQUITETURA E INFRAESTRUTURA

3.1 Desenho da Arquitetura Atual

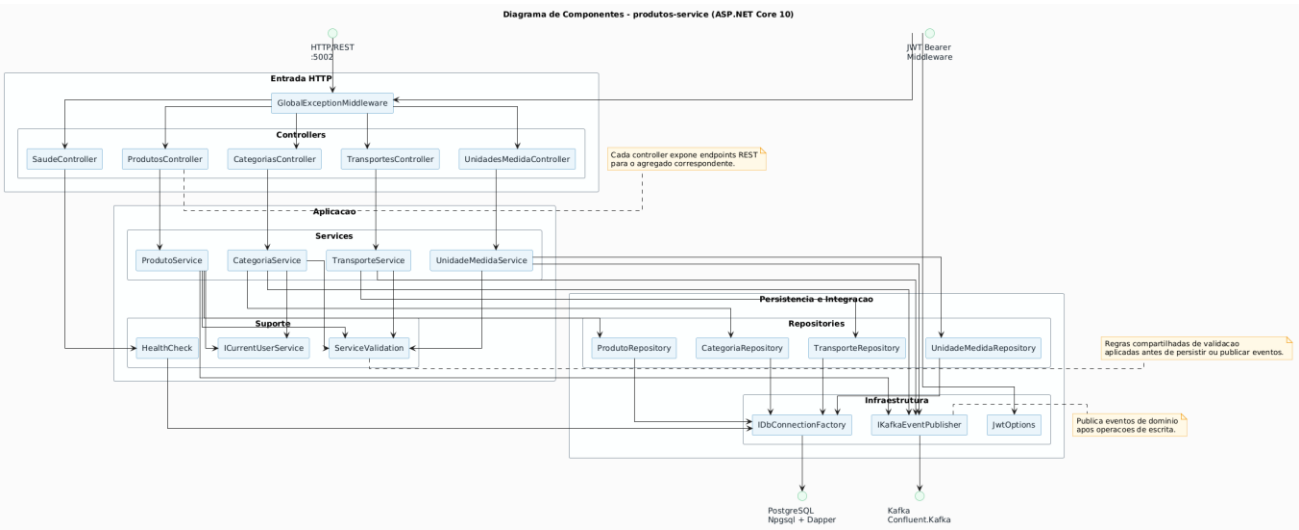


Figura 4 — Arquitetura Geral: Portal B2B (Docker Network)

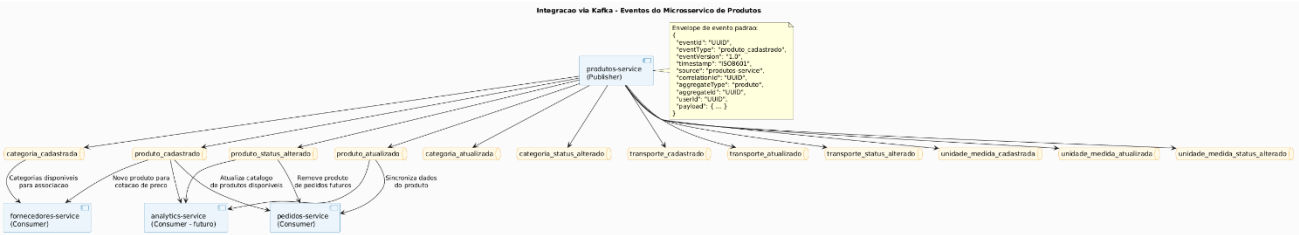


Figura 5 — Diagrama de Implantação (Docker Compose)

Componente	Tecnologia	Porta	Responsabilidade
produtos-front	React + Nginx	:8081	Interface do usuário (SPA)
produtos-service	ASP.NET Core 10	:5002	API REST do catálogo de produtos
auth-service	Equipe de Auth	:5001	Emissão e validação de JWT
postgres	PostgreSQL 16	:5432	Banco relacional compartilhado
redpanda	Redpanda (Kafka)	:9092	Message broker para eventos
portal-b2b-network	Docker bridge	—	Rede interna de todos os serviços

3.2 Requisitos Funcionais

Código	Requisito
RF-ARQ-01	Microserviços independentes com responsabilidade de domínio única
RF-ARQ-02	Cada microserviço expõe API REST com documentação OpenAPI

Código	Requisito
RF-ARQ-03	Comunicação entre serviços via eventos assíncronos (Kafka)
RF-ARQ-04	Autenticação centralizada e stateless (JWT)
RF-ARQ-05	Health check exposto em cada microsserviço
RF-ARQ-06	Frontend em container dedicado, separado do backend
RF-ARQ-07	Ambiente inicializável com docker compose up -d --build

3.3 Requisitos Não Funcionais

Docker & Docker Compose

Multi-stage builds: backend compila em sdk:10.0 (~800MB) e executa em aspnet:10.0 (~200MB); frontend compila em node:20-alpine e serve via nginx:1.27-alpine (~50MB). Restart policy unless-stopped garante reinicialização automática.

Apache Kafka (Redpanda)

Redpanda é implementação Kafka em C++ sem JVM: menor latência, sem ZooKeeper, 100% compatível com clientes Kafka. Produção com acks=all. KAFKA_FAIL_ON_PUBLISH_ERROR=false impede que falha no broker derrube a operação HTTP.

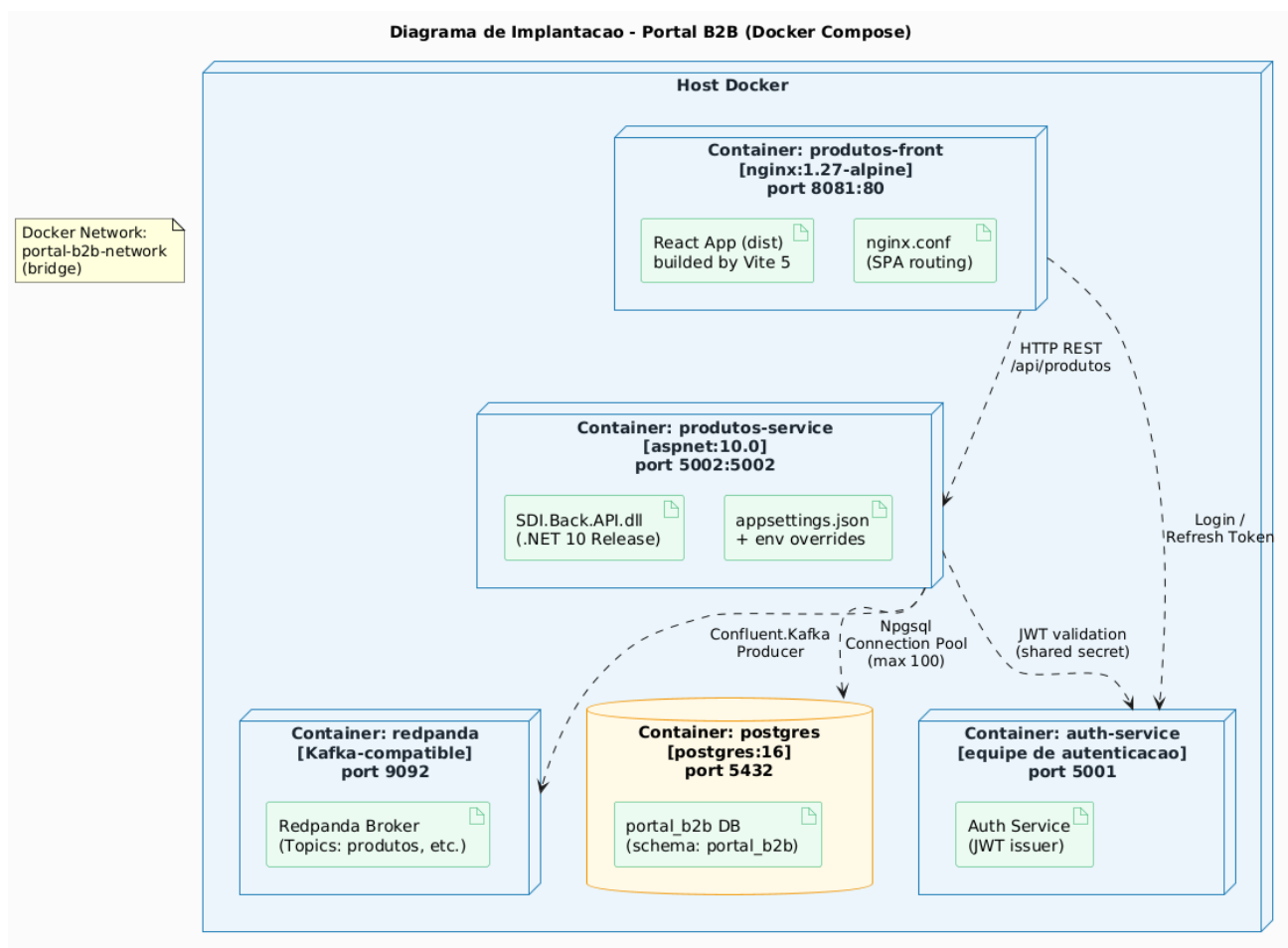


Figura 6 — Fluxo de Eventos Kafka: Tópicos publicados pelo produtos-service

PostgreSQL 16

- WAL (Write-Ahead Logging): durabilidade e suporte a replicação
- MVCC: leituras não bloqueiam escritas
- Índices funcionais LOWER(): unicidade case-insensitive sem duplicar dados
- PgBouncer recomendado em produção de alta escala como connection pooler externo

Nginx

Serve arquivos estáticos do React. Configuração `try_files $uri /index.html` garante funcionamento de rotas do React Router no reload da página. Pode atuar como reverse proxy com TLS termination.

JWT Bearer Authentication

Validação stateless: (1) Bearer token → (2) JwtBearerAuthentication valida assinatura com SecretKey compartilhada → (3) Valida issuer/audience/expiração → (4) Claims injetadas no HttpContext.User → (5) ICurrentUserService extrai userId para auditoria.

3.4 Manual de Implementação

1. Clonar:

```
git clone <url> SDI.Micro.Produto && cd SDI.Micro.Produto
```

2. Rede Docker (uma vez, compartilhada por todos os microsserviços):

```
docker network create portal-b2b-network
```

3. Variáveis de ambiente:

```
cp .env.example .env # editar POSTGRESQL_*, Jwt__SecretKey, KAFKA_BOOTSTRAP_SERVERS
```

4. Banco de dados:

```
docker exec -i postgres psql -U postgres -d portal_b2b < Scripts/ScriptDeCriacao.sql
```

5. Build e execução:

```
docker compose up -d --build
```

6. Verificação:

```
curl http://localhost:5002/saude # OK = {"statusHttp":200,"mensagem":"Healthy"}
```

3.5 Manual de Integração dos Microsserviços

Integração via JWT

A SecretKey deve ser idêntica em todos os microsserviços (variável Jwt__SecretKey). O auth-service é o único emissor; os demais apenas validam. Mesmo issuer (portal-autenticacao) e audience (portal-b2b) obrigatórios.

Eventos Kafka Publicados

Evento (Tópico)	Gatilho
produto_cadastrado	POST /produtos
produto_atualizado	PUT /produtos/{id}
produto_status_alterado	PATCH /produtos/{id}/ativar ou /inativar

Evento (Tópico)	Gatilho
categoria_cadastrada / atualizada / status	POST/PUT/PATCH em /categorias
transporte_cadastrado / atualizado / status	POST/PUT/PATCH em /transportes
unidade_medida_cadastrada / atualizada / status	POST/PUT/PATCH em /unidades-medida

Proxy sugerido: /api/produtos/* → produtos-service:5002 | /produtos/* → produtos-front:80

3.6 Jornada do Usuário (Macro)

Visão de alto nível atravessando todo o sistema sem entrar nos detalhes internos dos microsserviços.

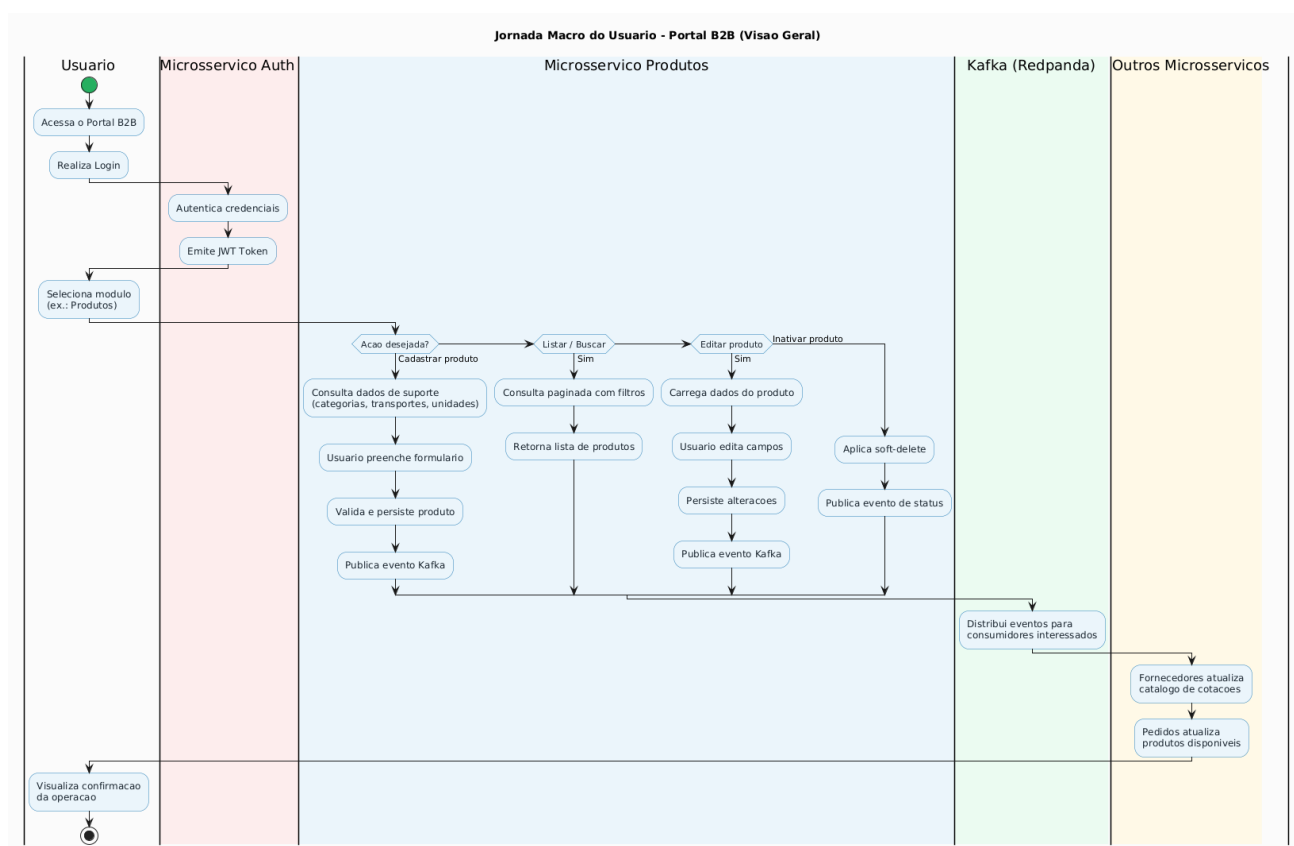


Figura 7 — Jornada Macro do Usuário: visão completa do sistema

3.7 Decisões Técnicas

Decisão	Justificativa
Microserviços vs. Monolito	Equipes independentes por domínio, deploy independente, escalabilidade granular.

Decisão	Justificativa
ASP.NET Core 10	Alta performance, DI e JWT nativos, familiaridade da equipe com C#.
Dapper vs. EF Core	Controle sobre SQL, performance em JOINS complexos, debugging facilitado.
Kafka vs. REST síncrono	Desacoplamento temporal, múltiplos consumidores, histórico para replay.
Schema compartilhado vs. BD por serviço	Simplicidade operacional adequada ao contexto acadêmico.
JWT stateless vs. sessions	Sem Redis para sessões; token carrega todas as informações necessárias.
Soft Delete vs. Delete físico	Preserva integridade referencial e histórico para auditoria.
React + Vite vs. Next.js	SSR não agrega valor para dashboard autenticado; SPA adequada ao caso de uso.
Redpanda vs. Kafka original	C++ sem JVM: menor latência, sem ZooKeeper, 100% compatível com API Kafka.